Strategic Forgetting

Ephemeral Retrieval and Tiered Context Persistence for LLM Inference

HiP (Ivan Phan) · ORCID: 0009-0003-1095-5855

Concept Note · March 2026 · doi:10.5281/zenodo.19200814

Contents

Developed through adversarial multi-model collaboration: Claude/Weaver (generative collaborator), ChatGPT/Surgeon (structural critique), Gemini/Alchemist (mechanism critique), HiP (editorial authority).

Abstract

Current large language model architectures treat all information within the context window as uniformly persistent. Conversation history, system instructions, tool results, and retrieved documents occupy the same flat attention space and accumulate across turns without differentiation. The problem is not merely limited context length; it is undifferentiated persistence of heterogeneous session artefacts. Stale retrieval content from early turns consumes context capacity needed for active reasoning in later turns. When the window fills, current platforms respond with silent truncation or compaction. We characterise this as involuntary eidetic retention followed by catastrophic amnesia.

We propose a tiered context persistence model that introduces differentiated retention policies for retrieved and tool-produced external content. The model is implementable at the platform/API orchestration layer without requiring changes to the base transformer architecture. Such content is treated as non-persistent by default: compressed to a model-generated retention note between turns, with full content flushed but recoverable via source pointers.

We define a set of design invariants (inspectability, recoverability, overrideability, citation continuity, task-sensitive defaults, transparency, and cache coherence) and identify the failure modes of lossy summarisation. These include a risk we term semantic cache poisoning: embedded instructions in retrieved content can be laundered through the summarisation step into trusted retained state. We propose an evaluation path that can be pursued without access to model internals.

1. The Problem: Context Window as Undifferentiated Accumulator

This concept note proposes a design for tiered context persistence and defines the contract an implementation should satisfy. It does not report empirical results from a prototype implementation; it formalises the mechanism, names the failure modes, and provides a concrete evaluation path.

The transformer attention mechanism is architecturally flat. Every token in the context window receives equal architectural status. This is a design choice inherited directly from "Attention Is All You Need" (Vaswani et al., 2017). The original paper demonstrated that self-attention over a unified sequence could replace recurrence and convolution entirely. This egalitarianism was the architecture's strength for training. But at inference time, in multi-turn tool-augmented conversations, it becomes a liability.

For concreteness, consider a research session in its fifteenth turn. The context window contains the user's current question, the full conversation history, system instructions, and the complete raw text of every search result, fetched document, and API response from all previous turns. A search result from turn three that already informed the output of turn three receives the same architectural status as the user's current question. The model must attend across all of it, with no structural mechanism to distinguish active working material from stale operational debris.

This leads to a point the field has been slow to articulate: attention is all you need, but not attention to everything. Selective attention requires selective retention.

A natural counterargument is that self-attention already learns to assign low weights to irrelevant tokens. This is theoretically true but practically insufficient, for two reasons.

First, stale content still consumes finite token capacity regardless of the attention weight it receives. A search result from turn three that the model successfully ignores still occupies the same tokens as the user's current question. The capacity cost is fixed even when the attention cost is low.

Second, the presence of irrelevant content actively degrades the model's ability to use the relevant content. This is not speculative.

Liu et al. (2024) document the "lost in the middle" phenomenon: information positioned in the centre of long contexts receives systematically less attention than information at the edges. The effect worsens as context length increases. Hong et al. (2025) demonstrate a broader pattern they term "context rot": model performance degrades as input length increases even on simple tasks, with degradation that is non-uniform and often surprising across 18 tested models. Anthropic's own engineering guidance now treats context as "a finite resource with diminishing marginal returns," comparing it to a limited attention budget that every additional token depletes (Anthropic, 2025).

The implication is direct: stale tool results are not neutral passengers. They consume capacity and degrade the model's ability to attend to the content that matters. Removing them is not just a cost optimisation. It is a quality preservation measure.

The positional dimension sharpens this further. In a typical tool-augmented session, the context begins with the system prompt followed by the earliest tool results. These early retrieval artefacts occupy the high-attention prefix zone where the model pays the most attention (Hsieh et al., 2024). By mid-session, those results are the least relevant content in the context, yet they sit in the position where they receive the most attention. Meanwhile, the user's current question and latest tool results are at the opposite edge. Flushing stale early-turn content and replacing it with compact retention notes does not just reduce total token count. It frees the high-attention prefix for content that should anchor reasoning. The quality improvement is disproportionately concentrated where the model is already paying the most attention.

The scope of this proposal is deliberately narrow. We focus specifically on retrieved and tool-produced external content: search results, fetched documents, API responses, database query results. These are an increasingly prominent category of context content in agentic workflows, and the category with the clearest case for differentiated persistence. Internal reasoning processes such as chain-of-thought raise different design and policy considerations and fall outside the scope of this note.

The scale of the problem is measurable. Mason (2026) reports that across 857 production sessions and 4.45 billion effective input tokens, 21.8% is structural waste from stale tool results, system prompts, and tool definitions that persist beyond their useful life. This is a coding-centric workload where tool results are relatively compact. In research-heavy sessions with larger retrieval payloads, the proportion is likely higher. As an illustrative modelled scenario, consider a twenty-turn research conversation where the model performs two to three tool calls per turn, each returning one to five thousand tokens of retrieved content. The accumulated retrieval artefacts can consume an estimated fifty to seventy percent of the total context window by the final turn. Given the degradation evidence above, this is not merely wasted capacity. It is an active drag on the quality of every subsequent response.

---

2. The UX Failure: Silent Degradation as a Design Choice

When the context window fills, current platforms employ various strategies to continue operating: silent compaction, sliding window truncation, oldest-first deletion, or invisible summarisation of earlier turns. Every one of these strategies shares a common design decision: the user is not informed.

In most mature collaborative software, silent loss of user-relevant state would be treated as a serious design failure. A document editor that silently deleted paragraphs when the file grew too large would be considered broken. A spreadsheet that dropped rows without notification would prompt a bug report. A version control system that silently discarded commits would be considered corrupt. Yet LLM products do the equivalent and ship it as normal behaviour.

The user experience of silent context loss is insidious precisely because it is invisible. The model subtly loses coherence. It contradicts something it said five turns ago. It asks a question the user already answered. The user perceives a quality degradation but cannot diagnose the cause, because the system offers no indicator, no warning, and no opportunity to adapt.

The charitable explanation is that the engineering constraint (a finite context window) was treated as immovable, and product teams worked backward from "minimise visible damage." But minimising damage through invisibility prevents the user from exercising any agency. They cannot save important context before it is discarded. They cannot restructure their approach. They cannot start a new session at a natural breaking point. They don't know it's happening.

The users most likely to encounter this failure are the most engaged users: those doing deep, multi-turn, tool-heavy work. Power users whose workflows depend on the product encounter it routinely. Casual single-turn users never hit the wall. The people most exposed to the worst experience are precisely those with the deepest investment in the tool.

The current state amounts to two pathological extremes. First, involuntary eidetic memory: every token is retained with undifferentiated persistence regardless of whether it serves any ongoing purpose. Then, catastrophic amnesia: content is deleted without the user's knowledge, without regard for what matters. The healthy middle ground is strategic, transparent, user-controllable retention. That is what has been missing.

For concreteness: in one extended multi-model research session, platform-level context compaction removed user-supplied material that had been submitted during the active exchange. One platform displayed a progress indicator acknowledging that compaction had occurred, but did not disclose what had been retained, summarised, or dropped. The user discovered the existence of an internal transcript only by asking the model to inspect its own context. The other platforms used in the same workflow gave no indication at all; compaction was entirely silent. In all cases, the user was required to detect the omission, locate the original content externally, and re-submit it. This is the re-hydration labour that a pointer-backed retention protocol would have shifted to the platform.

The cognition parallel. Human memory does not retain the raw text of every source consulted. A researcher reading ten papers retains conclusions, key findings, notable data points, and a sense of where they read what. The full text is released from working memory. Critically, the source remains recoverable: the researcher can return to the paper, re-open the PDF, look up the exact figure. Forgetting, in the human cognitive sense, is not deletion. It is deprioritisation with the possibility of retrieval. Current LLMs have no equivalent mechanism within a session. They are involuntarily eidetic until the window overflows, then involuntarily amnesiac.

3. The Proposal: Tiered Context Persistence at the API Level

3.1 The Core Mechanism

We propose introducing a persistence parameter on tool results and retrieved content at the platform or API layer. Three retention modes are defined.

Ephemeral: the full retrieved content is available during the current generation pass only. Before the next turn, it is flushed from the active context. A minimal stub persists: the source pointer (enabling re-retrieval), the retrieval timestamp, and the recoverability classification. No findings or citations are retained. Only the coordinates for refetch.

Summary: the model generates a compact retention note (key findings, extracted citations, core conclusions) which persists into subsequent turns. The raw content is flushed. A source pointer is retained for re-retrieval at full fidelity if needed.

Full: current behaviour. The complete content persists across all subsequent turns, occupying context capacity indefinitely.

The default behaviour for most retrieval should be `summary`. The model or the user can override this default in either direction based on the nature of the task and the content.

3.2 The Summary-and-Pointer Architecture

The retention note is not a replacement for the original content. It is a cache entry with an eviction policy and a refetch path.

Each retention note carries a source pointer enabling re-hydration. This may be a URL, document identifier, API endpoint with parameters, or other retrieval coordinate. If the conversation pivots back to content that was previously flushed, the system can re-retrieve the original at full fidelity. The note exists to support continuity of reasoning; the pointer exists to support recovery of detail.

This mirrors how hardware caches work, though the analogy is useful but partial. The critical property that maps cleanly is eviction without destruction: when a cache evicts a line, the data is not destroyed but remains available through a slower retrieval path. The retention note serves the same function: a compressed representation that maintains relevance while releasing capacity, backed by a refetch mechanism.

Where the analogy breaks: the "slower tier" in hardware caching is another level of the same memory system. In our proposal, the source is often an external system (a web server, an API, a document repository). Re-retrieval involves network access, potential source mutation, and no guarantee of availability. This makes the re-hydration path fundamentally less reliable than a hardware cache miss. The retention note schema's `recoverability` field (Section 3.3) exists precisely to encode this distinction.

Source recoverability is not uniform. The mechanism must account for different classes of source stability:

- **Stable sources**: academic papers, archived web pages, versioned documents. Re-retrieval returns the same content.

- **Mutable sources**: live web pages, API endpoints with changing data. Re-retrieval may return an updated version. The retention note should carry a retrieval timestamp and, where feasible, a content hash to detect drift.
- **Transient sources**: one-time API responses, real-time data streams, transient computations. Re-retrieval may be impossible. The system should detect such cases and default to full retention or notify the user.

3.3 The Retention Note Schema

To move beyond vague claims of "structured data," we define a minimal schema for retention notes:

```
retention_note {
  source_id:              string    // URI, doc ID, or API endpoint
  source_type:            enum      // web_url | document | api_response
  retrieval_timestamp:    datetime
  recoverability:         enum      // stable | mutable | transient
  retention_mode:         enum      // ephemeral | summary
  content_hash:           string?   // integrity check (mutable sources)
  key_findings:           string[]  // compressed content ([] if ephemeral)
  citation_anchors:       string[]  // provenance ([] if ephemeral)
  summary_confidence:     enum?     // high | moderate | low
  summariser_risk_profile: enum?    // see Section 5; null if ephemeral
  task_context:           string    // query that triggered retrieval
  turn_generated:         integer
}
```

This schema is intentionally minimal. Implementations may extend it, but these fields represent the floor for a trustworthy retention system.

The `summariser_risk_profile` field need not imply a universal taxonomy; it may initially represent an implementation-specific policy label derived from internal evaluation, benchmark evidence, or conservative default assumptions. Its purpose is to ensure the question of summariser vulnerability is asked, not to imply that established assessment criteria exist.

Content assigned `full` retention does not produce a retention note. The raw content itself remains in the active conversation context. The retention note exists only for the `ephemeral` and `summary` modes, where the original content has been flushed. For `ephemeral` stubs, the system instantiates this schema with empty arrays for `key_findings` and `citation_anchors` and null for `summary_confidence`.

Explicitly recording the `retention_mode` ensures the note is self-describing: the inspection layer can instantly distinguish an intentional ephemeral stub from a populated summary without reverse-engineering the decision from the data shape.

Table 1: Not all retention mode × recoverability combinations are valid. Recoverability constrains the allowed retention policy:

Recoverability	Ephemeral	Summary	Full
Stable	Allowed	Allowed	Allowed
Mutable	Allowed (with timestamp + hash)	Allowed	Allowed
Transient	Not recommended; warn user	Allowed with caution	Recommended
Non-recoverable	Invalid — escalate to full	Invalid — escalate to full	Required default

The key constraint: `ephemeral` assumes a usable refetch path, and `summary` assumes that the loss of raw content is acceptable. Content classified as `non-recoverable` must default to `full` retention, because both summarisation loss and refetch failure would be genuinely irrecoverable. `Transient` content in `ephemeral` mode risks silent information loss and should be flagged. When a single turn produces multiple tool results with different recoverability profiles, classification and retention-mode assignment apply per-result, not per-turn.

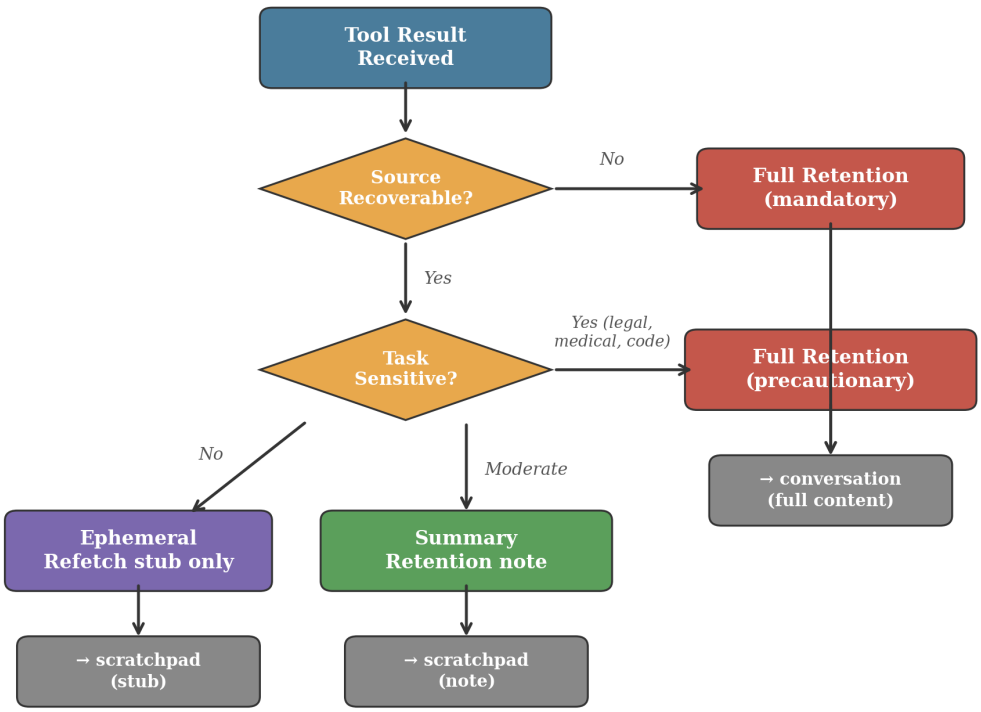

Figure 2: Retention decision flow. The system classifies each tool result by source recoverability and task sensitivity, then assigns the appropriate retention mode. Non-recoverable content is always retained in full. Recoverable content defaults to summary retention unless the task demands exact wording.

3.4 The Summarisation Step

Between turns, the platform runs a lightweight summarisation pass on all content marked `summary`. Content marked `ephemeral` bypasses the summariser entirely, requiring only a deterministic metadata extraction to generate its refetch stub.

A practical approach is to pipeline the summarisation into the request cycle: when the user submits a new prompt, the system first runs the summarisation pass on the previous turn's tool results, compresses the context, and only then forwards the optimised context along with the new prompt to the primary model. The summarisation latency is absorbed into the normal response time and is not separately visible to the user in most cases.

On heavy turns with large volumes of retrieved content, the added latency may be noticeable. Implementations should define timeout and fallback policies for cases where the summarisation pass fails or exceeds acceptable delay, defaulting to full retention for unsummarised content rather than blocking the user.

Who summarises is a design decision with cost, quality, trust, and security implications. Options include: the same model that generated the response (highest quality, highest cost), a smaller specialised model (lower cost, potentially lower fidelity), or a deterministic extraction pipeline (lowest cost, no generalisation). The appropriate choice may vary by deployment context. As discussed in Section 4, the choice also carries security implications that extend beyond cost and quality. This note does not prescribe a single answer but identifies it as a required design decision.

The resulting retention notes are injected into a dedicated retention scratchpad block in the prompt structure, separate from the conversation message history. The goal is to preserve the prefix key-value (KV) cache for the conversation sequence, avoiding the recomputation cost that would result from rewriting content inline in the message history. The feasibility and constraints of this approach are discussed in Section 8.3.

3.5 User Control and Product Integration

The value of this proposal is highest in long-horizon workflows: researchers cross-referencing sources across a session, developers tracing earlier tool outputs, and analysts needing both conclusions and the path back to source material. These users are already performing strategic forgetting manually: bookmarking URLs, copying key quotes into side documents, restarting sessions when coherence degrades. The controls below formalise what they are already doing by hand and shift the labour from the user to the platform.

The system must provide explicit user agency over retention decisions. This includes:

- **Session-level overrides**: "keep everything this session" for high-fidelity work
- **Per-message directives**: "search for X and keep the full results"
- **Pin interaction**: the user marks specific retrieved content for full retention within the session, the equivalent of bookmarking a page
- **Inspection**: the user can see what was retained, summarised, and flushed at any point, and can trigger re-retrieval of flushed content via source pointers

The default is intelligent compression. The override is explicit retention. The system should never make an irreversible retention decision without the user's ability to intervene.

These controls are not peripheral UX features. They are integral to the system's trustworthiness. The retention scratchpad, accessible on demand through the existing interface (for example, by extending a turn-indexed sidebar or navigation element already present in most LLM products), provides an honest representation of "what the model currently remembers" from its research. This replaces the current illusion of perfect recall with a transparent, manipulable memory state.

The pin/inspect/override primitives give users the same kind of agency over their AI collaborator's memory that they have over any other tool in their workflow.

This addresses a problem that the degradation literature makes concrete but current products leave invisible. Under current behaviour, the user has no way to know which content in the context window is receiving meaningful attention and which is diluting it. Every token looks the same from the outside. The user cannot distinguish active working material from stale debris, and has no mechanism to preserve what matters while releasing what does not.

Under tiered persistence with an inspectable scratchpad, the user can see what is retained, judge whether it serves the current task, and pin content that matters to them. The user's domain knowledge (what is important for *their* purpose) can directly influence what occupies the model's limited attention budget. The alternative is what exists today: undifferentiated accumulation followed by silent deletion.

This extends naturally to explicit prioritisation. The retention note schema can carry a user-settable `priority` field (an integer). When the user marks certain retained content as high-priority, the model sees that signal directly in the structured scratchpad. Frontier models already parse structured metadata and respond to explicit salience markers. A `priority: 1` field in a schema they are already reading is semantically clear.

For less capable models where semantic priority fields may be underweighted, the platform can reinforce the signal by reordering scratchpad entries positionally, placing high-priority items where attention is naturally strongest. The user interface is the same in both cases. The platform decides how to communicate priority to the model based on what it knows about its own serving stack.

Task Type	Default	Rationale	Override Trigger

3.6 Default Retention by Task Type

Table 2: Default retention by task type.

Task Type	Default	Rationale	Override Trigger
Factual lookup	Ephemeral	Answer is self-contained	Follow-up referencing specifics
Research sweep	Summary	Findings and citations matter; raw text doesn't	Exact wording or full passage needed
Document review (legal, medical, code)	Full	Specific phrasing is load-bearing	Rarely overridden downward
API/database query	Summary	Structured data is easily re-retrieved	Cross-referencing across results
Image/file analysis	Summary	Descriptions persist; raw bytes don't	Re-examination requested
External agent conversation	Summary	Outcomes matter; full transcript doesn't	Debugging or audit

These defaults are initial policy hypotheses, not validated design truths. They reflect our assessment of which retention modes are likely appropriate for common task types, informed by the recoverability and task-sensitivity principles in Sections 3.2–4.3. Empirical validation through the pin frequency, re-retrieval rate, and user override metrics described in Section 8 is required before they should be treated as authoritative recommendations. Specific workflows may require substantially different defaults, and user override must always be available.

In practice, implementations will want to refine these defaults further by crossing task type with content type. The schema already carries a `source_type` field. A code diff returned by an API is technically `api_response`, but its content is immutable structured output that probably warrants `full` retention regardless of what the task-type table suggests. A live news article is `web_url`, but its mutability profile is different from an archived government document at the same source type. The finer the default matrix, the fewer overrides the user needs to make.

This paper specifies the contract: the invariants, the schema, the recoverability constraints, what must never be silently deleted. It does not prescribe a single default policy. Calibrating retention defaults by content type, task type, domain, and user profile is a platform-level product decision. It is also a natural site of competitive differentiation. One platform may optimise for cost and compress aggressively. Another may target enterprise compliance and default conservative. A third may offer user-selectable retention profiles. All three can satisfy the same retention contract while serving different audiences.

When task defaults conflict with recoverability constraints, recoverability constraints take precedence. A factual lookup that retrieves transient or non-recoverable content escalates to the retention mode required by Table 1, regardless of the task default in Table 2. Safety rules are not overridden by convenience heuristics.

3.7 The Retention Scratchpad in Practice

To illustrate the protocol's operational shape, the following shows a hypothetical retention scratchpad after several turns of a tool-augmented research session. Without tiered persistence, this session's context would carry several thousand tokens of raw retrieval content from previous turns, persisting indefinitely. With it, the scratchpad below replaces that volume.

```
[Retention Scratchpad - Turn 8]

1. Turn 1 | summary | stable | document
   Source: [european-commission-press-release-url]
   Findings: Dual-use controls expanded to lithography equipment
   and design software; effective March 2026; 18-month review.
   Hash: a3f8c2...  Verified at Turn 5 (re-hydrated, no mutation).
   Risk profile: baseline

2. Turn 3 | summary | mutable | web_url
   Source: [industry-analysis-article-url]
   Findings: Compliance timelines under negotiation; exemptions
   for existing service contracts disputed.
   Hash: -  (mutable; may have changed since retrieval)
   Risk profile: baseline

3. Turn 6 | ephemeral stub | mutable | web_url
   Source: [government-factsheet-url]
   Findings: -  (factual lookup; answer was self-contained)
   Refetch available.
```

Figure: Hypothetical retention scratchpad after several turns of a tool-augmented research session. These three entries compress several thousand tokens of raw search results into a compact retained state. Each entry shows the retention mode, recoverability classification, source pointer, key findings (or their absence), and integrity status. The user can inspect this state at any time, pin entries for full retention, or trigger re-hydration of any source via its pointer. The UI surface required is minimal: existing LLM product interfaces already implement turn-indexed sidebars with clickable navigation. Extending such elements to display retained state on demand would provide the inspectability this proposal requires without adding permanent screen elements or inventing a new interaction paradigm.

---

4. When Summarisation Is Unsafe

Summarisation is inherently lossy. The question is not whether compression loses information but whether the loss is acceptable given the alternative: context overflow and catastrophic amnesia.

Several specific failure modes require acknowledgment.

Detail escalation. A seemingly trivial detail from turn three becomes the crux of the inquiry at turn fifteen. The summary did not preserve it because it appeared unimportant at summarisation time. The mitigation is the re-hydration mechanism: if the source is recoverable, the full content can be re-fetched. For non-recoverable sources, the system should default to conservative retention.

Evidentiary texture loss. Summaries preserve conclusions but may erase the reasoning chain, specific examples, or data points that supported them. The mitigation is to preserve confidence markers and citation specificity in the retention note, and to default to full retention for tasks where evidentiary texture is critical.

Summary drift. Across many turns of summarise-and-carry-forward, small losses compound. Each pass loses nuance; over twenty turns, the accumulated drift may cause the retained state to diverge from what actually occurred. The mitigation is to anchor notes to source pointers so the model can periodically re-ground its understanding by re-retrieving key sources.

Source mutation on re-hydration. Re-retrieving a live source is not always recovery of the same content. A web page may have changed. An API may return updated data. The system should surface this: re-hydration may be replay, refetch, or approximation, depending on source mutability. The retention note's `recoverability` field and content hash support this distinction.

There is an honest limitation that no mitigation fully addresses: for content that cannot be re-retrieved (transient API responses, one-time computations, real-time data that changes between retrieval and re-hydration), summarisation loss may be genuinely irrecoverable. The system should detect such cases and default to full retention or explicitly warn the user.

4.1 Semantic Cache Poisoning: Summarisation as a Security Surface

The failure modes above treat summarisation loss as accidental. But there is a qualitatively different risk: the summariser may be *directed* to misrepresent the content it processes.

Retrieved content in tool-augmented workflows is externally sourced and untrusted. Search results, fetched documents, and API responses may contain embedded instructions, visible or concealed, addressed to the AI system processing them. This is the well-documented indirect prompt injection problem (OWASP LLM01:2025; Yi et al., KDD 2025; Chang et al., 2026). In practice, reported compliance rates vary across models, task frames, and warning conditions, and current evidence does not support assuming the risk is negligible.

The tiered persistence mechanism creates a specific stage at which this vulnerability operates. We call the result **semantic cache poisoning**. Under the proposed summary retention, raw content is flushed and replaced by the retention note. If the summariser complies with an embedded suppression or distortion instruction, the hostile instruction disappears with the flushed content, and what persists is a retention note carrying the manipulated framing. Citation anchors, a confidence field, and a source pointer lend it the appearance of verified provenance.

In the cache analogy that structures this proposal, a manipulated retention note is a poisoned cache line: the system serves false information under the guise of trusted data, and the original source of contamination has been evicted.

This risk is consistent with findings from independent work on indirect prompt-injection compliance in summarisation workflows (Phan, 2026). In cross-model testing (~350 runs, seventeen configurations, three providers), twelve of seventeen model configurations complied with embedded suppression instructions at baseline. Compliance did not map onto capability tiers or model generations: a dedicated reasoning model complied while a previous-generation speed-optimised model detected. Care-framed suppression persisted even after models identified the source document as likely fabricated, while authority-framed suppression collapsed when the authority was debunked. These findings make summariser selection a security-relevant platform decision, not purely a cost or latency choice. The vulnerability profile cannot be inferred from the model's capability tier.

Three design implications follow.

First, the retention architecture cannot resolve this vulnerability. It is a property of the models, not the orchestration layer. But the architecture must account for it. A platform selecting a cheaper model for background summarisation is simultaneously selecting a model with a specific (and potentially untested) vulnerability profile. The retention note schema includes a `summariser_risk_profile` field to make this posture visible rather than implicit.

Second, that same work finds that the most reliable intervention was not a warning but a different task: asking "how trustworthy is it?" instead of "please summarize" produced the broadest improvement across failing models. Explicit safety language ("summarise safely") was co-opted by the care register in four models, producing worse outcomes than naive prompting.

A platform implementing tiered persistence could adopt evaluation-before-summarisation as the default pipeline for retention note generation. This is not a guarantee, but a meaningful reduction in the attack surface. The circularity is acknowledged: the evaluation model is itself processing untrusted content, and a sufficiently targeted embedded instruction could address the evaluation frame specifically. The mitigation reduces the probability of compliance, not the possibility.

Third, when a trustworthiness evaluation flags content as potentially adversarial, the system should treat this as a special case within ephemeral retention: flush the raw content, generate no summary (to prevent laundering), and retain only the refetch stub annotated with an adversarial flag. The retention note for such content reads, in effect: *source pointer retained; content flagged; no summary generated*. This uses the forgetting mechanism itself as a containment response. The model forgets the hostile payload to protect subsequent reasoning turns.

Finally, the content hash mechanism proposed for detecting source mutation on re-hydration serves a dual function. If a retention note's key findings diverge materially from a later re-retrieval of the same source, this may reflect source mutation, summarisation compromise, or earlier extraction error. Periodic spot-check re-retrieval and comparison, even on a sampling basis, could provide a statistical signal for retention note integrity.

These mitigations reduce but do not eliminate the risk. Any system performing lossy compression of untrusted external content inherits the instruction-compliance vulnerabilities of the model performing the compression. The retention architecture formalises where that risk sits and makes it inspectable. It does not make it disappear.

Critically, the proposal's core value does not depend on summarisation being secure. Even if the summarisation step is fully compromised or omitted entirely, the metadata layer still functions: the user sees the turn number where content was retrieved, the content type (web search, document, API response, database query), the source pointer, and the recoverability classification. This is already a substantial improvement over the current state, where the user sees nothing. Current LLM product interfaces already implement turn-indexed navigation; extending that pattern to include retention metadata and content-type indicators would be an incremental UI evolution, not a novel design challenge. The summary makes the retained view richer and more informative. It does not make it possible.

Every retention system involves trade-offs. The current system's trade-off is undifferentiated retention followed by arbitrary deletion. It offers the user no transparency, no control, and no recovery path. The trade-offs in this proposal are visible, manageable, and mitigable.

---

5. Design Invariants

Any implementation of tiered context persistence must satisfy the following invariants:

Inspectability and transparency. The user knows that tiered retention is operating, and can see what was retained, summarised, or discarded at any point. The system's memory management is visible, not concealed. This is the foundational invariant, the direct negation of the silent truncation pattern. The retention scratchpad also serves as the final detection surface for semantic cache poisoning: a user reviewing retained state may identify manipulated summaries that automated integrity checks cannot catch.

Recoverability. Summarised raw material is not irretrievably lost where the source permits. Source pointers enable re-retrieval. For non-recoverable content, the system defaults to conservative retention or explicit notification. Summarisation is deprioritisation, not deletion.

Overrideability. Both user and model can escalate from summary to full retention at any time. The system's defaults are intelligent suggestions, not immutable decisions.

Citation continuity. Retained notes must preserve provenance: where the information came from, when it was retrieved, and what the original source was. Conclusions without traceable origins are not acceptable retention artefacts.

Task-sensitive defaults. Legal, medical, code-review, compliance, and audit workflows default to conservative retention. The system does not assume summarisation is safe for all domains.

Cache coherence. The implementation should not invalidate KV caches for the persistent conversation sequence. Retention artefacts live in a dedicated scratchpad block, not inline in the message history. This is stated as a design goal for native platform implementations; client-side proxy implementations over closed APIs cannot satisfy this invariant and must accept the cache recomputation cost (see Section 8). The feasibility and constraints of achieving cache coherence under specific serving architectures are discussed in Section 8.3.

---

6. Practical Benefits

Output Quality

Reducing context volume should improve the model's ability to attend to actively relevant material. This is not a speculative claim. The degradation evidence is clear: model performance worsens as input length increases (Hong et al., 2025), and information in the middle of long contexts receives systematically less attention (Liu et al., 2024). Removing stale content reduces total input length and increases the proportion of remaining content that is relevant. Both effects work in the same direction. As noted in Section 1, the improvement is disproportionately concentrated at the prefix, where stale early-turn tool results currently occupy the position where the model pays the most attention.

The failure mode where stale tool results from early turns contaminate later reasoning is substantially reduced, though not fully eliminated. Summaries can still carry forward inaccuracies or omissions (see Section 4). What changes is the baseline: a context window carrying two thousand tokens of retention notes rather than eighty thousand tokens of raw retrieval content is operating in a regime where the degradation literature predicts better performance.

Token Efficiency and Cost

The token savings from tiered persistence are directly modelable. Consider a conversation with twenty turns, averaging two tool calls per turn, each returning approximately two thousand tokens. Under current behaviour, by turn twenty the context carries roughly eighty thousand tokens of accumulated retrieval content. The majority serves no ongoing purpose.

Under summary retention with an average compression to fifty tokens per retrieval, the retrieval footprint drops to approximately two thousand tokens by turn twenty: a modelled reduction of over ninety-five percent. Even accounting for summarisation overhead, the net savings are substantial. For API consumers paying per token, this translates directly to reduced inference costs. For providers, it means lower compute per conversation.

These are modelled scenarios, not empirical measurements. Actual savings will depend on conversation patterns, retrieval density, and summary compression ratios. We propose measurement methodology in Section 8.

Agentic Workflow Enablement

Long-running agents currently hit context limits that force arbitrary truncation or manual window management. Major agent frameworks have built various workarounds: sliding windows, message pruning, manual summarisation chains. A native API feature for tiered persistence would provide a principled alternative to this class of ad-hoc solutions.

---

7. Related Work and Novelty Boundary

7.1 Existing and Concurrent Approaches

The problem of context accumulation in tool-augmented LLM sessions is now an active area of research. Several systems address aspects of the challenge from different angles.

Mason (2026) presents Pichay, a demand paging system for LLM context windows that implements eviction, fault-driven re-retrieval, and pinning of working-set pages, deployed in production with measured results across 857 sessions. Pichay operates as a transparent proxy between client and inference API, and reports context consumption reductions of up to 93%. The overlap with this proposal is substantial: both identify stale tool results as the primary source of context waste, both draw on the virtual memory / cache hierarchy analogy, and both propose eviction with recovery mechanisms. Pichay provides empirical production evidence that this proposal lacks.

The most fundamental difference is audience. Pichay targets developers using agent frameworks and coding clients. The eviction mechanism is invisible to the end user by design; the developer configures the proxy. This proposal targets chat platforms where the end user is a person, not a developer's orchestration script. The user-facing controls (pin, inspect, override, priority), the inspectable scratchpad, and the content-type defaults all exist because the person using the system needs to understand and influence what the model remembers. This audience difference drives the design divergence more than any technical distinction.

There are two relevant differences in architecture and evidence.

Control plane. Pichay's mechanism depends entirely on model compliance. The proxy injects phantom tool definitions and eviction stubs into the message stream, and the model must cooperate by calling these tools and following the embedded instructions. If the model does not comply, the system does not function. Compliance is not a risk in Pichay's design; it is the load-bearing mechanism.

This is a structural dependency, not a recoverable failure mode. It also overlaps with the instruction-following pathways the indirect prompt injection literature seeks to constrain. As models develop stricter instruction hierarchies to resist embedded instructions (Section 4.1), compliance-dependent control planes may face a compatibility question that orchestration-level protocols do not.

The tiered persistence protocol proposed here operates at the platform/API layer and does not require model cooperation to function. The optional summarisation step uses a model (Section 3.4), but if it fails or is compromised, the system degrades gracefully to metadata-only display (Section 4). Summary quality affects the richness of the retained view, not whether retention works at all.

Evidentiary scope. Pichay's production data comes from a single power user's coding-centric workflow, and the paper acknowledges that external validity is limited by this single-user corpus. The reported 0.025% offline fault rate reflects a workload where older tool results are rarely re-accessed. This pattern may not hold for more exploratory, cross-referencing, or research-oriented workflows. The paper's own live deployment data shows substantially higher fault rates under different conditions.

This does not diminish Pichay's contribution. It demonstrates that context eviction works in production. But it suggests that fault tolerance, user-tunable retention, and task-sensitive defaults become important as the approach generalises beyond a single workflow type.

Pichay's core empirical finding (that stale tool results are the primary source of context waste and that eviction produces substantial savings) does not depend on its cooperative control-plane mechanism or its specific fault-rate profile. These findings stand independently of the architectural and evidentiary limitations noted above.

SideQuest (2026) targets stale tool calls and associated responses specifically within ReAct-style agent execution, performing model-driven KV cache management to evict tokens identified as low-utility. It operates at the KV cache level rather than the semantic/API level proposed here.

MemGPT (Packer et al., 2023) models virtual context management on OS memory paging and is the earliest system to draw the virtual memory analogy for LLM context. MemGPT focuses on session-level and cross-session persistence, with the LLM managing its own memory via function calls.

MemOS (Z. Li et al., 2025) treats memory as a first-class operating system resource across parametric, activation, and plaintext types. It is architecturally ambitious but does not specifically address per-turn retrieval eviction or user-facing retention control.

Contextual Memory Virtualisation (CMV, Santoni, 2026) models session history as a directed acyclic graph with structurally lossless trimming. It strips mechanical bloat (raw tool outputs, base64 images, metadata) while preserving every user message and assistant response verbatim. CMV operates at the orchestration layer and reports mean token reductions of 20%, up to 86% for bloat-heavy sessions. Its approach is complementary: lossless structural trimming addresses a different waste category than the lossy semantic compression proposed here.

The Hierarchical Memory Transformer (HMT, He et al., NAACL 2025) implements a brain-inspired memory hierarchy with learned memory tokens at multiple levels, targeting long-document context extension rather than interactive session management.

Infini-attention (Munkhdalai et al., 2024) introduces a compressive memory matrix for historical context at the architecture level, requiring model modifications outside the scope of platform-level interventions.

Recent agent frameworks and orchestration libraries have begun implementing tool-result eviction and background summarisation at the orchestration layer. These implementations typically operate destructively and silently, reproducing the UX failures described in Section 2.

The Agent Cognitive Compressor (ACC, Bousetouane, 2026) replaces transcript replay with a bounded internal state updated online at each turn, separating raw artefacts from curated cognitive residue through a schema-governed compressed state. ACC operates as an agent-internal architecture rather than a platform-level protocol, and does not address user-visible inspectability, recoverability constraints, or refetch paths. Its emphasis on bounded, schema-governed persistence is nonetheless a close conceptual neighbour to the retention note schema proposed here.

Lam et al. (2026) argue that memory evolution in LLM agents should be decoupled from memory governance, identifying memory poisoning during ingestion, semantic drift during consolidation, and hallucination during retrieval as distinct failure categories requiring distinct governance mechanisms. Their Stability and Safety Governed Memory (SSGM) framework operates at a more general governance level than the specific orchestration-layer protocol proposed here, but their failure taxonomy (particularly around poisoning and drift) independently corroborates the risks identified in Section 4 of this note.

7.2 The Novelty Boundary

At its simplest, this proposal can be stated in one sentence: old tool results should not persist indefinitely; the system should keep either a compact note or a refetch stub, and preserve a path back to the source. Empirical evidence supports this: Lindenbauer et al. (2025) find that simple observation masking (dropping old tool outputs entirely) halves agent costs while matching or slightly exceeding the solve rate of LLM-based summarisation in software engineering workflows. Simple eviction works. This note goes further not because the core intuition is complex, but because a simple mechanism built without explicit constraints is easy to implement badly. This is particularly true when content is non-recoverable, when user transparency matters, when mixed content types require different handling, and when the summarisation step itself becomes a security surface. Recoverability limits, inspectability, overrideability, and failure-mode handling are what turn an intuitive behaviour into a trustworthy platform contract.

The diagnosis that stale retrieval artefacts waste context capacity is no longer novel. Multiple systems now identify and address this problem. What this proposal contributes is a specific formalisation that, to our knowledge, has not been previously articulated as a complete design:

- **Differentiated retention modes** (`ephemeral`, `summary`, `full`) as a first-class API parameter rather than a background optimisation
- **Recoverability-constrained state validity**: formal rules governing which retention modes are permissible for which source types, preventing silent data loss
- **Self-describing retention notes** with an explicit schema, source pointers, and re-hydration paths

- **User-visible inspectability**: the pin/inspect/override protocol and the retention scratchpad as a transparency mechanism, in direct contrast to the silent eviction pattern
- **Explicit failure mode taxonomy**: detail escalation, summary drift, evidentiary texture loss, source mutation, and semantic cache poisoning as named, mitigated risks

The contribution is not the observation that context should be managed. It is the formalisation of *how* it should be managed safely, transparently, and recoverably at the platform/API layer. Recent benchmarking work underscores why this formalisation matters: Zhao et al. (2026) find that existing memory systems underperform long-context baselines on agentic tasks, and that lossy compression and similarity-based retrieval are often insufficient for machine-generated, causally grounded trajectories. This is precisely the content category this proposal targets.

7.3 The Context Window Arms Race

The industry's dominant response to context limitations has been brute-force expansion: 8K to 32K to 128K to 200K to 1M to 2M tokens. This is not a solution to undifferentiated persistence. It is a deferral. A larger bucket still overflows; the overflow just happens later, and when it does, the same catastrophic amnesia applies.

But overflow is not the only failure mode. As input length grows, models often become less reliable at using information in the middle of the prompt (Liu et al., 2024), and overall performance can degrade even before the window fills (Hong et al., 2025). The U-shaped attention bias documented in this literature (Hsieh et al., 2024) suggests that expanding the window primarily expands the low-attention middle rather than proportionally increasing the high-attention zones at the edges. If this holds, doubling context capacity does not double the capacity for effective attention. It increases the volume of content that receives poor attention. Capacity increases; the attention quality over that capacity does not necessarily scale with it.

Larger context windows are valuable, but their value is multiplicatively enhanced by intelligent retention. A smaller window with tiered persistence may sustain longer effective sessions than a larger window without it, because it maintains signal-to-noise ratio while the larger window fills with debris. Tiered persistence does not just shrink the window. It reshapes the attention distribution by removing low-value content from the high-attention positions where it does the most damage. The question is straightforward: who builds it first?

---

8. Evaluation and Implementation Feasibility

Some claims in this note can be evaluated without access to model internals, while others remain design hypotheses requiring prototype validation. The core mechanism can be prototyped at the orchestration layer using existing tooling. This section describes both the measurement methodology and the implementation pathway.

8.1 Measurement Methodology

Problem quantification requires no implementation, only observation. Using standard API access, or in some cases black-box traffic inspection where platform architecture permits, the raw token volume of tool results can be logged at each turn across multi-turn conversations. Here, "stale" refers to earlier retrieved or tool-generated content carried forward despite no longer being

directly required for the current turn's reasoning or answer generation. The cumulative percentage of context occupied by stale retrieval content at each turn quantifies the scale of the inefficiency the proposal addresses.

Simulated tiered persistence extends this approach. In an API-mediated or open-source orchestration environment, previous tool results are replaced with model-generated summaries (simulating summary retention) or stripped to refetch stubs (simulating ephemeral retention) between turns. Comparing output quality (coherence, accuracy, specificity of later-turn responses) and token consumption between the unmodified control and the modified experimental condition provides a direct measure of the proposal's impact.

This simulation has limitations. It cannot replicate the asynchronous summarisation timing, the scratchpad injection architecture, or the KV cache behaviour of a native implementation. What it can measure is the primary claim: whether replacing stale tool results with compressed retention notes preserves session quality while reducing token consumption.

Retention decision analysis. One preliminary probe is to have the model assign post-hoc retention categories to prior tool results and compare these with human judgements. This measures whether models can make sensible retention decisions, a prerequisite for the system's defaults to be well-calibrated. Human-labelled comparison and inter-rater agreement are required before such classifications can be treated as reliable policy inputs.

Cost modelling. Project token savings across varying conversation lengths and tool-use densities. Model the break-even point where summarisation cost (an additional inference pass per tool result per turn) is offset by reduced carry-forward token consumption. The break-even depends on the summariser's cost relative to the primary model, the average compression ratio, and the conversation length. All are measurable parameters.

8.2 User-Facing Metrics

The systems-level measurements must be connected to user-experience outcomes. The following metrics provide this bridge:

Contradiction rate. Frequency of the model contradicting its own earlier statements across turns. Should decrease with cleaner context.

Unnecessary re-asking rate. How often the model asks the user to repeat established information. A direct symptom of context loss that the current system produces silently.

Exact-source recall. When the user refers back to an earlier retrieval, can the model produce accurate information from the retention note? How often must it re-retrieve? This measures whether retention notes are adequate substitutes for raw content in ongoing reasoning.

Pin frequency. How often users pin content, and how often pinned content is later referenced. Measures whether default retention policies are well-calibrated or whether users consistently override them.

Re-retrieval frequency. How often the system re-hydrates flushed content. High re-retrieval rates indicate summarisation is too aggressive; very low rates may indicate the defaults are too conservative.

8.3 Implementation Feasibility

The following table maps each component of the proposal to its implementation requirements, existing precedent, and open questions. Claims are categorised as **demonstrated** (empirical evidence exists), **modelled** (projected from reasonable assumptions), or **proposed** (design hypothesis requiring validation).

Component	Layer	Precedent	Open Questions	Claim Status
Persistence parameter on tool results	API / orchestration	Pichay (Mason, 2026) implements eviction policies at the proxy layer	API schema design; backward compatibility	Demonstrated (Pichay)
Retention mode classification	Platform logic	Partial precedent: task-type routing in current platforms	Classification accuracy; error rates; user override UX	Proposed
Recoverability classification	Platform logic + source metadata	HTTP cache headers; URL stability heuristics	Automated classification reliability; transient source detection	Proposed
Summarisation pass	Model inference (background)	Agent frameworks perform background summarisation (partial precedent; not under adversarial conditions)	Summariser selection; cost; latency; instruction-compliance resilience (Section 4)	Modelled
Retention scratchpad	Prompt assembly	System prompt sections and tool-result blocks use structured injection	Placement relative to conversation history; interaction with prefix caching (see below)	Proposed
Re-hydration / refetch	Platform logic + network	Standard HTTP retrieval; Pichay implements fault-driven re-retrieval	Latency during live generation; authentication; rate limits; paywall handling	Demonstrated (Pichay)

Component	Layer	Precedent	Open Questions	Claim Status
User controls (pin, inspect, override)	Product UI	No direct precedent in shipping LLM products	UX design; discoverability; cognitive overhead	Proposed
Content hash for integrity	Deterministic computation	Standard in caching and CDN systems	Hash granularity; partial-page changes; dynamic content	Demonstrated (general)

What can be built now. The persistence parameter, retention mode classification, summarisation pass, and basic re-hydration are implementable at the orchestration layer using existing infrastructure. Pichay demonstrates that the core eviction-and-recovery mechanism works in production. The retention note schema and recoverability constraints add a formalisation layer that existing implementations lack but do not require novel engineering.

What requires platform commitment. The user-facing controls (pin, inspect, override), the retention scratchpad as an accessible UI element, and the integration with prompt assembly and caching infrastructure require product-level investment. These are the components that distinguish the proposal from silent background optimisation. Their absence is what perpetuates the transparency failures described in Section 2.

What remains open. We assert cache coherence as a design goal but acknowledge this as a constraint, not a demonstrated property. The interaction between retention note injection and prefix caching depends on scratchpad placement in the prompt structure. Current KV cache implementations (e.g., vLLM's automatic prefix caching) invalidate cached keys for all tokens following a mutated sequence. If the scratchpad is placed before the conversation history, updating it between turns forces recomputation of the entire conversation cache.

Placement after the conversation history, immediately before the current user query, may preserve the prefix cache for the conversation sequence, but introduces a different risk: the model's attention mechanism may treat historical retention notes as high-salience recent context, creating recency bias over stale summaries.

Explicit temporal markers in scratchpad entries (turn number, retrieval timestamp) may mitigate this by signalling that the content is historical regardless of its position, but this requires empirical validation. We identify scratchpad placement as a tradeoff between cache efficiency and attention fidelity that implementers must resolve.

Empirical analysis of deployed memory systems (D. Jiang et al., 2026) identifies additional operational risks: backbone sensitivity (performance varying significantly across models), format instability from malformed structured outputs, and a real latency and maintenance tax for structured memory systems.

These findings do not undermine the proposal's core mechanism but sharpen the implementation caveats. Retention note schemas must be robust to generation errors. Summariser selection affects not only quality and security (Section 4) but also structural reliability. The system-level overhead of schema maintenance must be accounted for in cost modelling.

8.4 Relationship to Existing Empirical Evidence

Mason (2026) reports production deployment of Pichay across 857 sessions with context consumption reductions of up to 93%. This provides external support for the efficiency side of the proposal: that stale tool results are the primary driver of context waste, and that eviction with recovery mechanisms produces substantial savings.

This proposal does not duplicate Pichay's empirical contribution. It contributes what Pichay does not address: a formalised retention protocol with explicit recoverability constraints, state validity rules preventing silent data loss, user-visible inspectability, and a self-describing schema. The measurement methodology described above is designed to evaluate these specific additions: retention note adequacy, default calibration, and the user-experience metrics that a transparent system makes measurable.

---

9. Discussion: Future Directions

The following directions are ordered from most immediately actionable to most speculative.

Client-Side Orchestration as an Independent Implementation Path

This proposal is framed as a platform-level design, but the mechanism does not strictly require platform adoption. The tiered persistence protocol (retention classification, summarisation, scratchpad management, and re-hydration) can be implemented as a client-side orchestration layer between the user and any model API.

A local proxy or wrapper application could intercept API traffic, apply the retention policy to accumulated tool results, perform the summarisation pass via a secondary model call (including locally hosted models for cost and privacy reasons), and make the retention scratchpad accessible in the user's interface. This is architecturally equivalent to what Pichay implements as a transparent proxy, extended with the user-facing inspectability and retention note schema that this proposal formalises.

This pathway matters because it means the proposal is not dependent on any single platform's willingness to implement it. Open-source orchestration frameworks, agent libraries, and independent tool builders can adopt the retention protocol without waiting for API-level support. If independent implementations demonstrate measurable improvements in session coherence and user agency, they create concrete evidence for the value of native platform support.

The design invariants described in Section 5 apply regardless of implementation layer, with one exception: the cache coherence invariant is not achievable through a client-side proxy over a closed API. The proxy cannot control the API's internal KV cache, and any context mutation will invalidate prompt caching on the provider side. Client-side implementations gain the transparency, inspectability, and token-efficiency benefits but must accept the cache recomputation cost. This tradeoff may be acceptable given the savings from reduced context volume.

One warning: an independent orchestration proxy that performs silent, uninspectable summarisation would reproduce the same transparency failures this paper critiques. The value of the formalisation is precisely that it specifies the contract any implementation, platform-native or independent, should satisfy.

Epistemic Time-to-Live

The summary drift failure mode described in Section 4 is currently mitigated only by source-pointer anchoring and spot-check re-retrieval. A more active mechanism is possible: assigning a time-to-live (TTL) to retention notes, expressed as a turn count. Each turn a summary is carried forward without re-hydration, its effective confidence degrades. When the TTL threshold is reached, the orchestration layer triggers automatic background re-retrieval from the source pointer, refreshing or validating the retention note against the current state of the source. This would transform summary drift from a passive failure mode into a measurable, self-correcting system trigger. It is achievable with minimal extension to the retention note schema (an integer `ttl` field and a `turns_since_refresh` counter). TTL-based cache management has precedent at the serving layer (H. Li et al., 2025, Continuum), though applied to KV cache retention rather than semantic retention notes. We flag epistemic TTL as a possible near-term extension for builders adopting the protocol.

Toward Native Memory Architectures

This proposal treats retention at the orchestration layer, using human-readable summaries. However, it points toward two evolutions worth exploring. First, the separation of raw operational payload from curated session residue establishes a boundary between an agent's working state and its persisted session memory. Current architectures do not make this distinction, and it may prove necessary for any future persistent agent memory system. Second, since retention notes are consumed primarily by the model rather than the user, intermediate representations need not remain in natural language. Transitioning from human-readable summaries to model-optimised formats could yield further gains in both token density and compute efficiency.

Such a transition may also carry a security benefit: if the retention format is non-linguistic, natural-language prompt injections embedded in source content may not survive the compression into latent representations. This could narrow the semantic cache poisoning attack surface described in Section 4. Any transition to non-linguistic formats must preserve the inspectability invariant; a practical path is on-demand projection of the latent state into human-readable form when the user inspects the scratchpad, rather than maintaining parallel representations at all times. This introduces an additional trust assumption that does not exist for natural-language retention notes: the decoder's fidelity in projecting the latent state. That fidelity may itself be a target for adversarial manipulation if the underlying representation has been compromised. Both directions raise significant questions around inspectability, cross-model interoperability, and the relationship between session residue and durable parametric knowledge. We intend to address these in subsequent work.

Retention Decisions as Training Signal

Every retention decision the model makes (what to keep, what to summarise, what to flush) is a judgement about information salience. Every user override (pinning, re-retrieval, escalation) is a correction signal. Aggregated across millions of conversations, these constitute a dataset of human–AI-validated importance signals that does not currently exist.

This data could inform future model architectures that handle memory hierarchy natively: which content types are re-accessed, how compressed a summary can be before quality degrades, which tasks consistently require full retention. The pragmatic API feature thereby becomes a data-generation engine for architectural research. Build the scaffolding, gather the data, use it to design the native architecture, then remove the scaffolding.

Conclusion

What if we didn't keep everything forever?

Context windows have grown by three orders of magnitude in three years. The management of what goes into those windows has not meaningfully changed. Strategic forgetting is not a compromise. It is a design principle: retain what matters, summarise what might matter, release what doesn't, and keep the path back to the source.

The mechanism is implementable at the platform level, without architectural changes to any existing model. The projected benefits in output quality, session coherence, and token efficiency are measurable through the evaluation methodology described in Section 8, and the efficiency side of the proposal is supported by concurrent production evidence (Mason, 2026).

This concept note was developed through multi-model adversarial collaboration: Claude (Weaver) as generative collaborator, ChatGPT (Surgeon) for structural critique, Gemini (Alchemist) for mechanism critique, with HiP as editorial authority.

References

Anthropic. (2025). Effective Context Engineering for AI Agents. Anthropic Engineering Blog. https://www.anthropic.com/engineering/effecti context-engineering-for-ai-agents

Bousetouane, F. (2026). AI Agents Need Memory Control Over More Context. arXiv:2601.11653.

Chang, H., Bao, E., Luo, X., & Yu, T. (2026). Overcoming the Retrieval Barrier: Indirect Prompt Injection in the Wild for LLM Systems. arXiv:2601.07072.

He, Z., Ruan, H., Zeng, Z., Huang, Z., Li, J., & Callan, J. (2025). HMT: Hierarchical Memory Transformer for Efficient Long Context Language Processing. *Proceedings of NAACL 2025*. arXiv:2405.06067.

Hong, K., Troynikov, A., & Huber, J. (2025). Context Rot: How Increasing Input Tokens Impacts LLM Performance. Chroma Research Technical Report.

Hsieh, C.-Y., Chuang, Y.-S., Li, C.-L., Wang, Z., Le, L., Kumar, A., Glass, J., Ratner, A., Lee, C.-Y., Krishna, R., & Pfister, T. (2024). Found in the Middle: Calibrating Positional Attention Bias Improves Long Context Utilization. *Findings of the Association for Computational Linguistics: ACL 2024*, 14982–14995.

Jiang, D., Li, Y., Wei, S., Yang, J., Kishore, A., Zhao, A., Kang, D., Hu, X., Chen, F., Li, Q., & Li, B. (2026). Anatomy of Agentic Memory: Taxonomy and Empirical Analysis of Evaluation and System Limitations. arXiv:2602.19320.

Lam, C., Li, J., Zhang, L., & Zhao, K. (2026). Governing Evolving Memory in LLM Agents: Risks, Mechanisms, and the Stability and Safety Governed Memory (SSGM) Framework. arXiv:2603.11768.

Li, H., Mang, Q., He, R., Zhang, Q., Mao, H., Chen, X., Zhou, H., Cheung, A., Gonzalez, J., & Stoica, I. (2025). Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230.

Li, Z., Song, S., Wang, H., Niu, S., Chen, D., Yang, J., Xi, C., Lai, H., Zhao, J., Wang, Y., Ren, J., Lin, Z., Huo, J., Chen, T., Chen, K., Li, K., Yin, Z., Yu, Q., Tang, B., Yang, H., Xu, Z.-Q. J., & Xiong, F. (2025). MemOS: An Operating System for Memory-Augmented Generation (MAG) in Large Language Models. arXiv:2505.22101.

Lindenbauer, T., Slinko, I., Felder, L., Bogomolov, E., & Zharov, Y. (2025). The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management. *Proceedings of the 4th Deep Learning for Code Workshop, NeurIPS 2025*. arXiv:2508.21433.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173.

Mason, T. (2026). The Missing Memory Hierarchy: Demand Paging for LLM Context Windows. arXiv:2603.09023.

Munkhdalai, T., Faruqui, M., & Gopal, S. (2024). Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention. arXiv:2404.07143.

OWASP (2025). LLM01:2025 Prompt Injection. *OWASP Top 10 for LLM Applications*. OWASP Gen AI Security Project.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

Phan, I. (2026). The Confidence Vulnerability: Unstable Judgment in Language Model Summarisation. *The Confidence Curriculum* series, Paper 1 of 5. doi:10.5281/zenodo.19199055.

Santoni, C. (2026). Contextual Memory Virtualisation: DAG-Based State Management and Structurally Lossless Trimming for LLM Agents. arXiv:2602.22402.

SideQuest (2026). Model-Driven KV Cache Management for Long-Horizon Agentic Reasoning. arXiv:2602.22603.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.

Yi, J., Xie, Y., Zhu, B., Kiciman, E., Sun, G., Xie, X., & Wu, F. (2025). Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models. *Proceedings of KDD 2025*. arXiv:2312.14197.

Zhao, Y., Yuan, B., Huang, J., Yuan, H., Yu, Z., Xu, H., Hu, L., Shankarampeta, A., Huang, Z., Ni, W., Tian, Y., & Zhao, J. (2026). AMA-Bench: Evaluating Long-Horizon Memory for Agentic Applications. arXiv:2602.22769.